

Ingeniería del software de componentes y sistemas distribuidos

PEC2 - CD/CI, DevOps y cultura ágil: buenas prácticas para el desarrollo de software distribuido

Índice

Compromiso.....	2
Ejercicio 1 (25 %).....	3
Ejercicio 2 (25 %).....	4
Cuestión 2.1 (12,5 %).....	4
Cuestión 2.2 (12,5 %).....	5
Ejercicio 3 (25 %).....	6
Cuestión 3.1 (12,5 %).....	6
Cuestión 3.2 (12,5 %).....	6
Ejercicio 4 (25 %).....	8
Cuestión 4.1 (8,4 %).....	8
Cuestión 4.2 (8,3 %).....	10
Cuestión 4.3 (8,3 %).....	11

Compromiso

Certifico que he realizado la PEC 2 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento “Originalidad en la evaluación” disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de D.

Ejercicio 1 (25 %)

Al inicio de esta actividad os hemos propuesto el siguiente debate: “Debate los objetivos básicos que persigue una cultura DevOps y los mecanismos, herramientas y/o estrategias para conseguirlos.”. En este ejercicio os pedimos que escojáis uno de los objetivos que hayáis debatido en el espacio “Debate de la Actividad 5” del aula para explicar cómo lo aplicarías al proyecto **Photo&Film4You**.

Nota: Debéis indicar la entrada del debate donde se explique el objetivo elegido.



José Romero Nieto

22 de jun 10:11 | Última respuesta 22 de jun 19:16

Hola a todos.

Considero que uno de los objetivos más importante e interesante de la cultura DevOps es la importancia de reducir el tamaño de los lotes. La idea de hacer despliegues frecuentes (varias veces al día) en lugar de despliegues grandes y poco frecuentes (semanales o mensuales) quizás no reciba tanta atención como otras prácticas de DevOps. Este es un objetivo dentro del primer principio fundamental de DevOps (First Way), cuya intención principal es acelerar el flujo de trabajo de izquierda a derecha (desde desarrollo hasta operaciones).

Me parece importante porque mi experiencia personal en desarrollo de software me dice que:

- Grandes despliegues implican más riesgo de errores y de detección más difícil. No es lo mismo analizar unos pocos cambios que todo el código de un servicio.
- Feedback más lento. Cuanto más grande el despliegue más tardará el equipo de operaciones en devolver feedback, tanto porqué le lleva más analizarlo, como por que le llegan los cambios más tarde
- Posibilidad de generar cuellos de botella. Debido a que los equipos de operaciones reciben grandes cantidades de trabajo de golpe.
- Más tensión y riesgo. Cuando los despliegues son pequeños son menos problemáticos y ante un fallo es más fácil identificarlo y revertirlo.

¿Cuál es vuestra opinión? ¿Consideráis los despliegues pequeños un objetivo “clave”?

Entrada del debate donde se explica el objetivo elegido

En base al objetivo DevOps que propone José Romero Nieto sobre reducir el tamaño de los lotes, podemos aplicarlo al proyecto desplegando los pequeños cambios que hagamos en cada uno de los microservicios. De esta forma, como bien indica en su experiencia, mitigamos el riesgo de generar cuellos de botella, reducimos la propensión a errores y facilitamos la identificación de los errores al tratarse de unidades de código pequeñas e identificables.

Por ejemplo, si desplegamos un cambio realizado en el microservicio de *Course*, sólo nos haría falta desplegar dicho microservicio con sus respectivos *tests*, sin necesidad de tocar ni desplegar el resto. Cabe destacar que, para confiar en esta metodología y estar tranquilos a la hora de trabajar con ella, es necesario tener una *suite* de *tests* que cubra la mayor parte del código con tal de no introducir ningún *bug* que pase desapercibido.

Ejercicio 2 (25 %)

En la PRAC2 hicisteis la implementación de los microservicios *Course*, *Microcredentials* y *Notification* de **Photo&Film4You** mientras que en la PRAC3 desarrollasteis un conjunto de pruebas unitarias, de contrato e incluso de mutación para estos microservicios. Responded brevemente a las siguientes cuestiones:

Cuestión 2.1 (12,5 %)

La integración continua es la base para conseguir llevar software de calidad a producción de forma ágil. Indicad qué estrategia creéis que es la más óptima para el desarrollo, despliegue y puesta en producción de estos microservicios en un entorno de *Continuous Delivery*. Razonad la respuesta.

Para la estrategia de desarrollo, podemos seguir el **Trunk-Based Development** respaldado por una **pipeline** de despliegue en varias etapas.

En el *Trunk-Based Development*, todo el código, *scripts* de despliegue, configuración y pruebas de los tres microservicios debe integrarse en un único repositorio o rama principal (dependiendo de cómo lo planteemos) varias veces al día. Asimismo, se prohíbe el uso de ramas de larga duración, ya que estas provocan fases de integración largas e impredecibles al final del desarrollo. Como vimos en la primera cuestión, realizar integraciones con frecuencia nos obliga a realizar cambios pequeños y, consecuentemente, a reducir los *merge conflicts*.

Para hacer que el *feedback* de los cambios sea rápido y exhaustivo, se puede introducir una *pipeline* de validación en dos fases:

- **Fase de confirmación:** Debe ser rápida y ejecutarse al hacer un *commit*. Al hacerlo, el servidor de CI compila los microservicios y ejecuta las pruebas más rápidas (en este caso, las unitarias y de mutación de la PRAC3). Si pasa, generaría, en este caso, un contenedor desplegable.
- **Fase de aceptación:** Coge los contenedores de la etapa anterior y los despliega en un entorno de preproducción. Aquí se ejecutarán las pruebas que consumen más tiempo para comprobar que los microservicios se comunican correctamente sin introducir regresiones.

Para evitar ejecuciones innecesarias de la *pipeline*, es recomendable que los desarrolladores ejecuten los *tests* de la fase de confirmación en sus propias máquinas antes de enviar los cambios al repositorio central para evitar, posteriormente, arreglar los errores detectados en la *pipeline* y hacer que se ejecute dos o más veces.

Cuestión 2.2 (12,5 %)

¿Habéis seguido esta estrategia en el desarrollo de la asignatura? Razonad la respuesta.

Personalmente, he seguido esta estrategia de forma parcial. Aunque llevo tiempo trabajando como desarrollador en entornos que favorecen la cultura DevOps y la automatización, el flujo de trabajo que apliqué se desvió de algunos principios de integración continua descritos en la teoría.

Aplicado al proyecto Photo&Film4You, creé un repositorio Git durante el desarrollo de los microservicios de la PRAC2. Sin embargo, utilicé un modelo basado en ramas, las cuales eran `main` y `develop`. Iba implementando los requisitos en la rama `develop` mediante *commits* y, posteriormente, creaba una *pull request* (PR) en GitHub para fusionar el trabajo hacia la rama principal.

Analizándolo desde una perspectiva teórica, este enfoque entra en conflicto con el *Trunk-Based Development* propugnado en el libro *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Tal y como establece la literatura sobre integración continua, el uso de ramas de desarrollo prolongadas como `develop` retrasa el momento de la integración real. Al aislar el código, se incumple la premisa de que es imposible hacer integración continua “real” mientras se usan ramas, ya que se pierde el feedback inmediato y se retrasa la resolución de posibles conflictos hasta el momento de aprobar la PR.

Respecto a la *pipeline* comentada en la [cuestión anterior](#), el escenario ideal para conectar la PRAC2 y la PRAC3 habría sido implementar una *pipeline* de validación. Dicha *pipeline* en dos fases habría actuado como escudo ante cualquier intento de integración. Al ejecutarse correctamente, superando todos los *tests* de la PRAC3, la *pipeline* permitiría integrar los cambios de los microservicios y el *software* estaría siempre listo para desplegarse.

Ejercicio 3 (25 %)

En la PRAC2, los microservicios se desplegaron usando un patrón de despliegue con un formato propio dependiente del lenguaje, mientras que la infraestructura de soporte se desplegaba con una estrategia basada en el uso de contenedores.

Cuestión 3.1 (12,5 %)

Analiza dos inconvenientes del patrón de despliegue basado con un formato propio dependiente del lenguaje que os podéis encontrar al usar este patrón para desplegar los microservicios en el proyecto **Photo&Film4You**.

Por un lado, nos encontramos con la falta de portabilidad y la dependencia con el entorno de ejecución, ya que un empaquetado dependiente del lenguaje obliga al servidor destino tener instalada y configurada la versión exacta del entorno de ejecución. En este caso, se requeriría una versión específica del Java JDK, junto con las variables de entorno y librerías del sistema operativo necesarias. Así se dificulta la homogeneización de los entornos de desarrollo, pruebas y producción, por lo que aumenta la probabilidad de sufrir errores del tipo “en mi máquina local sí funcionaba” a la hora de pasar a producción.

Por otro lado, identificamos la dificultad para gestionar una infraestructura tecnológica heterogénea. A diferencia de los contenedores, los cuales ofrecen una forma estándar de desplegar cualquier aplicación sin importar lo que haya dentro, este patrón rompe la estandarización y encarece el mantenimiento operativo. Además, va en contra de una de las grandes ventajas de los microservicios, que es poder elegir el lenguaje de programación que mejor se adapte a cada problema. Por ejemplo, si necesitamos añadir en el futuro un microservicio en Python para procesar imágenes en Photo&Film4You, usar formatos dependientes del lenguaje hará que cada *script* de arranque, herramienta de despliegue y mecanismo de monitorización sea distinto para cada tecnología.

Cuestión 3.2 (12,5 %)

Considerando un escenario de alta carga donde el microservicio de notificaciones requiere un escalado masivo debido a un aumento repentino en la emisión de microcredenciales, ¿cuál sería el patrón de despliegue óptimo? Razona la propuesta desde una perspectiva de arquitectura.

Para afrontar este escenario, el patrón de despliegue óptimo es el de un servicio por contenedor (*Service Instance per Container*) gestionado por un orquestador de contenedores como Kubernetes.

Desde una perspectiva de arquitectura, esta propuesta se justifica por la escalabilidad horizontal que proporciona debido a que, al empaquetar el microservicio de notificaciones en un contenedor de Docker, un orquestador como Kubernetes puede utilizar herramientas para escalar automáticamente y horizontalmente un *pod* (unidad de computación más pequeña y básica que se puede crear y gestionar). De esta forma, podemos monitorizar métricas en tiempo real y levantar nuevas réplicas del microservicio en *pods* en cuestión de segundos para paliar la carga. Cuando el pico haya pasado, podemos destruir los *pods* sin problema con el fin de optimizar el coste de infraestructura.

Otro aspecto a destacar es que, a diferencia de desplegar una máquina virtual completa por cada nueva instancia, los contenedores comparten el núcleo del sistema operativo raíz. Por lo tanto, son muy livianos y su tiempo de arranque es mínimo, lo cual es una característica importante cuando el aumento de peticiones desde el microservicio de *Microcredentials* es repentino y el sistema debe reaccionar casi de inmediato.

A pesar de que la PRAC2 ya partía de esta base, es importante enfatizar que, para que este patrón de despliegue sea 100 % efectivo en este escenario, la arquitectura de comunicación debe ser **asíncrona y basada en eventos** utilizando un *message broker* como Kafka o RabbitMQ. De este modo, *Microcredentials* encola los eventos masivos y las instancias contenerizadas de *Notification* actúan como consumidores concurrentes, procesando así la carga a su propio ritmo sin saturar la red ni bloquear el *host*.

Ejercicio 4 (25 %)

Tras la implementación de los microservicios en la PRAC2 y la definición de las pruebas en la PRAC3, el equipo de **Photo&Film4You** quiere automatizar su entrega mediante una *pipeline* de despliegue. Esta *pipeline* se define como la manifestación automatizada del proceso que lleva el *software* desde el control de versiones hasta las manos del usuario.

La construcción del proyecto se lanza automáticamente a partir de la subida de nuevas funcionalidades al repositorio de código fuente por parte de los desarrolladores y se ha definido con estas cinco fases:

1. **Fase de construcción:** La *pipeline* se inicia automáticamente tras cada *commit* del desarrollador en el repositorio.
2. **Fase de confirmación:** Se realizan las validaciones técnicas iniciales y se crea el candidato a liberación.
3. **Fase de aceptación:** Se valida que el *software* cumple los requisitos funcionales y no funcionales del cliente.
4. **Fase de testeo manual y de UAT:** El *software* se despliega en un entorno similar a producción para pruebas exploratorias y validación de negocio.
5. **Fase de despliegue a producción:** El paso final de puesta en producción para entregar valor al usuario. En este caso el despliegue requiere una aprobación manual.

Cuestión 4.1 (8,4 %)

En la PRAC3 se definieron algunos de los cinco niveles de test típicos (unitarios, contrato, integración, componente y E2E). Sitúa cada uno de estos tipos de *test* en una de las fases del pipeline de despliegue si queremos obtener *feedback* y desestimar versiones que no pasen alguno de los *tests* lo más rápido posible. Razonad la respuesta.

Una *pipeline* de despliegue debe ejecutar primero las pruebas más rápidas y aisladas. Si estas fallan, detenemos el proceso inmediatamente y ahorramos tiempo y recursos. Solo si el código pasa los requisitos y pruebas impuestas, invertimos el tiempo en ejecutar las pruebas más lentas y complejas. En base a esto, asignamos cada tipo de *test* a las fases de la *pipeline* de Photo&Film4You:

1. **Fase de confirmación:** En esta fase necesitamos un *feedback* casi inmediato. Si algo falla aquí, la versión se descarta de inmediato. Aquí encontramos estos *tests*:
 - **Tests unitarios:** Prueban métodos o clases de forma aislada. Si el código no funciona, es inviable seguir con el resto de *tests*. Además, se ejecutan rápidamente.
 - **Tests de contrato:** En una arquitectura de microservicios, los tests de contrato validan que las “firmas” de las APIs entre consumidor y proveedor son compatibles sin necesidad de levantar toda la infraestructura. Como utilizan *mocks* predefinidos y se ejecutan muy rápido, deben ir en esta fase para detectar integraciones rotas antes de armar la aplicación.

2. **Fase de aceptación:** Si la *build* supera la fase de confirmación, se genera el candidato a liberación y se despliega en un entorno similar a producción, o si existe, en un entorno de integración. Aquí entran las pruebas automatizadas más costosas en tiempo y recursos, tales como las siguientes:

- **Tests de componente:** Validan el comportamiento de un microservicio entero o varios componentes juntos interactuando con su base de datos real. Suelen *mockear* los servicios externos. Como necesitan levantar la aplicación y la base de datos, tardan más y se sitúan al inicio de esta fase.
- **Tests de integración:** Verifican que un microservicio se comunica correctamente con dependencias reales, como bases de datos externas, colas de mensajería (Kafka en el caso de Photo&Film4You) o llamadas reales a otros microservicios. Suelen ser más lentos y frágiles por los problemas de red.
- **Tests E2E:** Prueban el flujo completo del sistema tal y como lo haría el usuario, empezando desde el *frontend* y terminando en el *backend* y bases de datos. Son las pruebas más lentas, costosas de mantener y propensas a *flaky tests*. Se ejecutan al final de la fase, cuando estamos seguros de que los componentes y las integraciones individuales funcionan bien.

El resto de fases no las usamos por los siguientes motivos:

- **Fase de construcción:** Es el detonante que da paso a la fase de confirmación.
- **Fase de testeo manual y de UAT:** Como indica el propio libro de *Continuous Delivery*, esta fase está reservada para el trabajo de valor humano, como pruebas exploratorias, validación de usabilidad y *User Acceptance Testing* por parte de los *stakeholders* de negocio. Las pruebas automatizadas como las unitarias, E2E, contrato, etc. ya deben haber probado que el *software* funciona correctamente antes de hacer perder el tiempo a un *tester* manual.
- **Fase de despliegue a producción:** Es la culminación de la *pipeline* tras la aprobación manual. En esta fase se aplican técnicas de *smoke testing* en producción, pero las *suites* de *tests* automatizados mencionadas ya deben haberse ejecutado con anterioridad.

Cuestión 4.2 (8,3 %)

¿En qué fase deben generarse los ejecutables de cada microservicio y por qué es un antipatrón volver a compilarlo o recrearlo en pasos posteriores? Razonad la respuesta.

Los ejecutables (.jar o las imágenes de los contenedores Docker de cada microservicio) deben generarse en la **fase de confirmación**, ya que en esta es donde el servidor de CI compila el código fuente, se ejecutan las pruebas de confirmación y, si pasan, se empaqueta el ejecutable. Dicho ejecutable se convierte en el candidato a liberación y se almacena en un repositorio de artefactos, como Nexus, Artifactory o Docker Registry. A partir de este punto, todas las fases posteriores se limitan a descargar y ejecutar el artefacto inyectándole la configuración propia del entorno, pero sin alterar el ejecutable.

En este escenario, recompilar el ejecutable en fases posteriores es un antipatrón que desvirtúa el propósito principal de la *pipeline* de despliegue. Al hacerlo, la fiabilidad de las pruebas se convierte en nula, ya que si validamos un ejecutable **A** en los entornos de aceptación y UAT, pero generamos un nuevo ejecutable **B** para producción, estamos desplegando un artefacto diferente. Como resultado, no podemos comprobar si el sistema funciona correctamente, ya que la versión **B** que llega a producción nunca ha sido testeada.

Además, esto conlleva el riesgo de generar el ejecutable usando una versión distinta del compilador de Java en otro servidor u otras desalineaciones, como con el gestor de dependencias, que podría generar una versión completamente distinta usando otras versiones de las dependencias o incluso otras dependencias en caso de tener configurado otro repositorio.

Por último, compilar, empaquetar y descargar dependencias consume tiempo y CPU. Hacerlo en cada etapa hace que la *pipeline* sea ineficiente y vulnere el principio de *feedback* rápido.

Cuestión 4.3 (8,3 %)

Si los responsables de Photo&Film4You deciden pasar de *Continuous Delivery* a *Continuous Deployment*, ¿cómo afectaría esto al flujo de trabajo entre las fases 4 y 5? Razonad la respuesta.

Este paso supone un cambio de paradigma que afectaría a la existencia y ejecución de las fases 4 y 5 de la *pipeline*. En esencia, la diferencia principal entre ambas prácticas radica en la intervención manual. En la *Continuous Delivery* el *software* siempre está listo para producción pero el despliegue final es una decisión de negocio que requiere que un humano apruebe el paso; por el contrario, en el *Continuous Deployment* cualquier cambio que revise con éxito la *suite* de *tests* automatizados se despliega en producción automáticamente sin ninguna interacción humana.

Por tanto, el flujo de trabajo se vería afectado de la siguiente manera para cada fase:

- **Eliminación de la fase 4:** En *Continuous Deployment*, no puede existir ninguna fase manual que bloquee la *pipeline*. El testeo exploratorio y el UAT siguen aportando valor, pero se trasladan a etapas posteriores a la liberación. Para conseguirlo de forma segura, el equipo debe usar técnicas como *Feature Flags* o *Dark Launches* que permiten desplegar el código en producción manteniendo la nueva funcionalidad oculta a los usuarios reales hasta que el negocio la valide manualmente en ese entorno.
- **Automatización de la fase 5:** La fase 5 pierde su requisito de aprobación manual. Se convierte en un *script* automatizado que se lanza de forma automática en el momento en el que la fase 3 finaliza su ejecución correctamente.

A modo de resumen, si el equipo de Photo&Film4You decide implementar *Continuous Deployment*, la ruta desde el *commit* hasta la entrega de valor al usuario se vuelve 100% automatizada. El éxito en la fase 3 desencadena la fase 5, lo cual se traduce en postergar las aprobaciones humanas y el UAT del proceso de despliegue del ejecutable.